**DevOps Infrastructure – Unit 1 Notes (Part 6)**

**Docker Object Types and Docker Layering & Filesystem**

---

## 1. Docker Object Types

Docker works using different object types that represent various components of containerized applications.

---

### 1.1 Containers

Definition:
A container is a running instance of a Docker image.

Key points:

- It is lightweight and isolated
- Runs a specific application or service
- Has its own filesystem, network, and processes
- Can be started, stopped, deleted, or restarted

Example:
When you run:
docker run nginx

A container is created from the nginx image and starts running.

---

### 1.2 Images

Definition:
An image is a read-only template used to create containers.

Key points:

- Contains application code and dependencies
- Immutable in nature
- Built using a Dockerfile
- Can be versioned using tags

Example:
nginx:latest
python:3.10

---

**1.3 Networks**

Definition:
Docker networks allow containers to communicate with each other and with the outside world.

Types of Docker Networks:

1. Bridge Network (default)
    - Used for communication between containers on the same host
2. Host Network
    - Container shares host network directly
3. Overlay Network
    - Used in multi-host environments (like Kubernetes or Docker Swarm)
4. None
    - No network access

Key benefits:

- Isolation of communication
- Secure inter-container connectivity
- Service discovery

---

**1.4 Volumes**

Definition:
Volumes are used to persist data generated by containers.

Problem:
Containers are temporary. When deleted, data is lost.

Solution:
Volumes store data outside the container lifecycle.

Key points:

- Data persists even after container deletion
- Can be shared between containers
- Managed by Docker

Example:
docker run -v mydata:/app/data my_app

---

## 2. Docker Layering and Filesystem

---

### 2.1 Docker Layering Concept

Docker images are built using multiple layers.

Each instruction in a Dockerfile creates a new layer.

Example Dockerfile:

```
FROM ubuntu
RUN apt-get update
RUN apt-get install python3
COPY app.py /app/
```

Each line above creates a separate layer.

---

### 2.2 Why Layering is Important

1. Reusability
   Layers can be reused across images
2. Faster Builds
   Only changed layers are rebuilt
3. Efficient Storage
   Common layers are shared between images

---

### 2.3 Docker Filesystem Structure

Docker uses a layered filesystem.

It consists of:

1. Read-only layers (image layers)

2. Writable layer (container layer)

---

**2.4 Copy-on-Write Mechanism**

When a container runs:

- It does not modify the original image
- A new writable layer is added on top

Behavior:

- Read operations → from image layers
- Write operations → stored in container layer

---

**2.5 Example Scenario**

If 3 containers are created from the same image:

- All share the same base image layers
- Each has its own writable layer

This reduces memory and storage usage.

---

**2.6 Storage Drivers**

Docker uses storage drivers to manage filesystem layers.

Examples:

- overlay2 (most common)
- aufs (older)
- btrfs

These drivers handle how layers are stacked and accessed.

---

**3. Summary**

- Containers are running instances of images
- Images are read-only templates
- Networks enable communication

- Volumes provide persistent storage
- Docker layering improves efficiency and reuse
- Copy-on-write allows containers to run without modifying base images